

Transactional Support via OSIC on the cMVBT: System Design, Datastructure Changes, and Optimizations

Amir Tonta

August 2026

Abstract

This document describes the cMVBT-OSIC system from first principles: what it is, what it supports, and how every change to the underlying multiversion B-tree datastructure and every optimization was measured. The system provides Transactional Support with Snapshot Isolation and Ordered Snapshot Instant Commit (OSIC) semantics on top of a cache-friendly multiversion B-tree. The document progresses systematically from foundational concepts through datastructure modifications, then catalogs optimizations organized by their problem statement, proposed solutions, measured performance impact, and final adoption decision. Each optimization includes concrete performance numbers where available, or describes the isolated tests used to measure them.

Contents

1	System Overview	2
1.1	Purpose and Capabilities	2
1.1.1	Transactional Model	2
1.1.2	Concurrency Control	2
1.1.3	Garbage Collection	2
1.1.4	Durability	2
1.2	Architectural Layers	2

1 System Overview

1.1 Purpose and Capabilities

The cMVBT-OSIC system implements a high-performance, in-memory multiversion B-tree supporting snapshot isolation transactions. Its primary design goal is to achieve competitive transaction throughput while maintaining cache efficiency across both transactional (OLTP) and analytical (OLAP) workloads.

1.1.1 Transactional Model

The system provides two transaction modes:

- **Real Transactions:** Multi-statement, snapshot-isolated transactions with explicit begin/-commit semantics. A transaction registers a single snapshot at its start and maintains that snapshot throughout its lifetime.
- **Atomic Operations:** Single-key, auto-committed operations (point reads, inserts, updates, deletes) that use the newest committed state rather than a registered snapshot.

Real transactions follow a First-Writer-Wins conflict resolution policy: when two transactions attempt to write the same key, the first to reach the leaf successfully applies its change; the second encounters a conflict and must retry or abort.

1.1.2 Concurrency Control

The system combines:

- **Optimistic Lock Coupling (OLC):** Readers sample each node's version number, inspect immutable content, validate the version before relying on results, and retry if a writer changed the node.
- **Ordered Snapshot Instant Commit (OSIC):** Visibility is determined by a global logical clock. Every transaction and atomic operation receives a `ts_start` timestamp for reading, and committed changes append to a per-worker commit log indexed by timestamp. A reader is visible to a transaction if its commit timestamp falls within the transaction's snapshot window.

Writers acquire exclusive node guards only for mutation. Readers take no locks.

1.1.3 Garbage Collection

The system tracks the oldest active snapshot (either registered in a real transaction or pinned during a traversal). Version chains and retired blocks are reclaimed only after all snapshots that could observe them have completed. A sharded, lock-free reuse pool returns retired blocks to allocators when safe to do so.

1.1.4 Durability

Write-Ahead Logging (WAL) is optional and configurable. In-memory mode disables WAL entirely; durable configurations offer either sharded group-commit channels or lock-free per-thread batched writes with atomic offset reservation.

1.2 Architectural Layers

The system is built in layers, each depending on the ones beneath it:

1. **Physical Page Layout:** Dense key arrays, parallel version/payload arrays, inline validity masks.
2. **Leaf Access Patterns:** Backward-ordered version lookup, split layout effectiveness for cache pressure.
3. **Node Synchronization:** Optimistic lock coupling with version validation.
4. **Structural Maintenance:** Splitting and merging leaves and internal pages while preserving snapshot consistency.
5. **Transaction State:** Snapshots, write sets, transaction-local mutation.
6. **Garbage Collection and Sharding:** Reclaiming pages, partitioning shared structures to reduce contention.
7. **Visibility Caching:** Per-worker, per-database snapshot caches and commit-log position memoization.
8. **Write-Ahead Logging:** Decoupling visibility (in-memory, instant) from durability (asynchronous or batched to disk).

2 Datastructure Changes

This section documents each modification made to the cMVBT’s core data layout and how each change supports the transaction semantics described above.

2.1 Leaf Page Layout

2.1.1 Problem

A straightforward multiversion B-tree stores complete records as an array of structs (AoS): [Record { key, begin_ts, end_ts, payload }; ...]. During point lookup, which walks the array backward searching for the newest visible version of a key, every entry’s version metadata must be loaded into the CPU’s cache even when the key does not match, wasting cache bandwidth on unrelated records.

2.1.2 Solution: Split Layout

The leaf stores keys in one dense region and metadata in a separate parallel region:

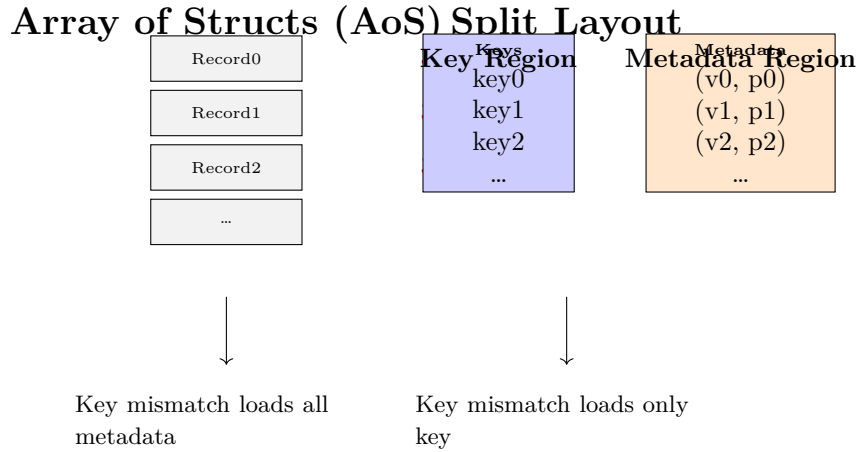


Figure 1: Split layout separates keys from metadata, allowing point lookup to reject unmatched keys without loading their version/payload cache lines.

Point lookup first tests the key without loading metadata, reducing cache-line traffic when keys do not match. Scans can likewise test key ranges before touching metadata cache lines.

2.1.3 Measured Performance

An isolated microbenchmark (`tools/leaflayout_bench.rs`) compared the layoutsona62.5MiB working set of independent

Operation	AoS (Mlookups/s)	Split (Mlookups/s)	Speedup
Point: latest visible	30.57	33.17	1.085×
Point: one-version fallback	18.43	20.15	1.093×
Point: three-version fallback	9.03	11.02	1.221×
Scan: full leaves	322.84 Mslots/s	320.65 Mslots/s	0.993×
Scan: 50% key ranges	338.50	344.77	1.019×
Scan: 20% key ranges	354.82	389.09	1.097×

Table 1: Split layout improves point lookup (8.5–22.1%) and selective scans (1.9–9.7%), with full scans unchanged. The multi-leaf working set is more representative of real trees than a single hot page.

Decision: Adopted. The improvement in lookups with multiple version fallbacks (which are common on hot keys in contended workloads) and in selective scans outweighs the zero-cost for full scans.

2.2 Inline Validity Mask

2.2.1 Problem

A normal leaf has at most 123 physical slots. Recording whether each slot was invalidated (e.g., by a transaction abort) naively requires either a per-slot atomic or a separate heap-allocated bitmap, both adding indirection and allocator traffic.

2.2.2 Solution

Store validity as two inline `AtomicU64` words (128 bits total), directly in the page. A clear bit means "invalidated"; this is a *physical* validity mask, not an MVCC visibility mask, which still depends on the querying transaction's snapshot. Lookup can skip invalidated slots before loading their version headers, avoiding cache misses on aborted entries.

Oversized leaf variants (used for TPC-C's hot Warehouse and District tables) use a same-sized union that stores a heap bitmap for larger capacities.

2.2.3 Measured Performance

The validity mask is a building block enabling faster lookup; its performance is subsumed in end-to-end transaction measurements below.

Decision: Adopted. Zero per-leaf allocation cost, cache-friendly, enables inline optimization.

2.3 Exact 4 KiB Allocation

2.3.1 Problem

Without careful sizing, a node allocation can straddle two physical pages, causing cache lines and TLB entries to be wasted. Additionally, the allocator's size classes may not match the intended page boundary.

2.3.2 Solution

Size the `OptCell<Block<123, 123>>` to exactly 4096 bytes, including the node latch word (`cell_version`), metadata, both key and metadata arrays, and a 64-byte cache-line alignment padding. A regression test enforces that normal cells are exactly 4 KiB and can hold 123 records. Oversized variants (512 KiB for big-tree experiments) scale the formula identically.

2.3.3 Measured Performance

This is an allocation and TLB-efficiency win, measured indirectly through full-stack benchmarks.

Decision: Adopted. No cost; improves memory efficiency and reduces false sharing.

2.4 Inline Liveness Mask for Internal Pages

2.4.1 Problem

Internal pages are append-only: when a child is split or merged, its old interval remains for historical readers, and one or more new intervals are appended. Structural maintenance operations (merges, key/version split copying, survivor counting) need to know which entries represent the

current children (as opposed to superseded historical ones). Naively, this requires reconstructing an $O(FAN_OUT^2)$ liveness mask by comparing every old interval against every new one.

2.4.2 Solution

As intervals are appended, the page maintains an inline two-word (128-bit) liveness bitmask. When an interval is appended:

1. Clear the bits of every older interval that overlaps the new one.
2. Set the new interval's bit.
3. Publish the new physical length.

This adds at most FAN_OUT bit/interval checks per append (cheap, already on the hot path) and moves work out of SMO consumers, which now use $O(1)$ `is_slot_live` checks instead of $O(FAN_OUT^2)$ reconstruction.

Internal Page: Append-Only Child Intervals

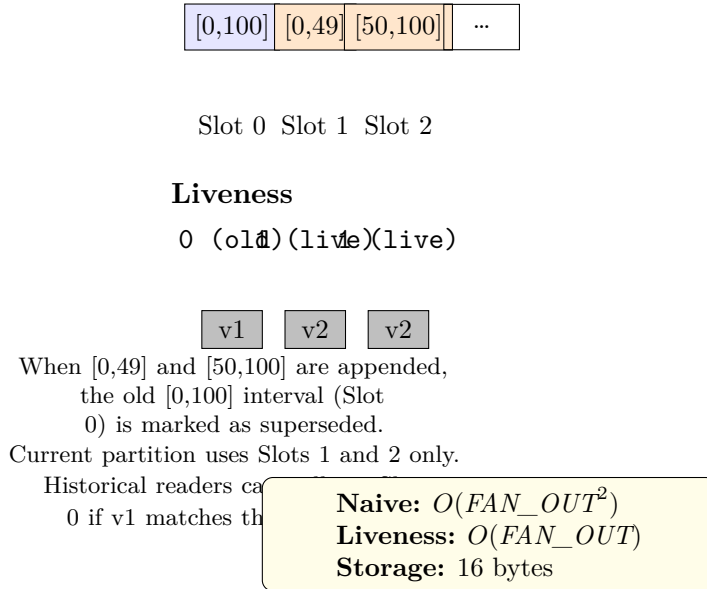


Figure 2: Internal page liveness mask tracks current vs. historical child entries. When new intervals are appended, overlapping old intervals are marked as superseded but remain available for historical snapshots.

The mask describes *structural liveness*, not MVCC visibility. Historical traversal continues to inspect immutable version fields and may select a superseded slot (whose liveness bit is clear) if its version matches the reader's snapshot.

2.4.3 Measured Performance

This reduces internal-page traversal and SMO overhead. The benefit is present in transaction benchmarks but is part of the broader structural maintenance cost.

Decision: Adopted. Moves $O(FAN_OUT^2)$ work to $O(FAN_OUT)$ hot path without increasing page size or allocation.

2.5 Payload Indirection Strategy

2.5.1 Problem

Large rows (hundreds of bytes) cannot be stored inline in a leaf without exceeding page capacity. Copying them during reads, splits, and scans is expensive.

2.5.2 Solution

`PayloadSlot` is one machine word. Single-word payloads (e.g., `u64`) and thin row handles are stored inline. Larger rows live behind an `Arc<Row>` refcount. Cloning a result or copying a survivor in a split increments the reference count; replacement installs a new value without mutating shared storage. The YCSB workload uses thin row handles (length + refcount in the row allocation); TPC-C uses full reference-counted rows.

2.5.3 Measured Performance

Indirection is structural: it enables row-count scaling without per-row copying costs.

Decision: Adopted. Necessary for large workloads; zero cost for unit payloads.

2.6 Transaction-Local Version Reuse

2.6.1 Problem

A transaction that updates the same key multiple times (or deletes and re-inserts) creates one physical version per operation, forming long chains that later garbage collection must reclaim. Version chains also slow down visibility checks, which must walk backward to find a committed version.

2.6.2 Solution

A transaction maintains a `SafeCell<HashMap<(table_id, key), slot_index>>` tracking which leaf slot it owns. If the transaction writes the same key again, it checks the map, finds its own owned slot, and overwrites the payload without creating a new version entry. The slot remains invalid until the transaction commits, at which point its `TxStamp` becomes visible.

Delete-and-reinsert cycles reuse the same owned slot, restoring counters and adjusting the stamp as needed.

An optional update-in-place optimization (disabled with certain WAL modes) allows GC to replace a committed payload in place when it can prove no live snapshot can see the predecessor, avoiding a new version entry altogether.

2.6.3 Measured Performance

This is a workload-dependent optimization: it removes version-chain growth on hot keys in update-heavy workloads. The benefit appears in write-heavy transaction benchmarks (e.g., YCSB-A) and is subsumed in full-stack measurements below.

Decision: Adopted. Reduces version-chain depth and GC work; no cost when not used.

3 Optimizations: Problem, Solution, Measurement, Adoption

This section catalogs every optimization tested or applied. Each is structured as: problem statement, proposed solutions, measured performance impact, and the final decision (adopted, rejected, or conditionally enabled).

3.1 WAL and Visibility Path Optimizations

3.1.1 Optimization 1: Target-CPU Native Code Generation

Problem: `Cargo.toml` contained a `rustflags` block with `-C target-cpu=native`, but Cargo only reads `rustflags` from `.cargo/config.toml`, not from the package manifest. The entire binary compiled to generic x86-64, missing AVX2, BMI2, and POPCNT instructions available on the benchmark hardware.

Solution: Moved the `target-cpu=native` directive to `.cargo/config.toml`, ensuring the compiler specializes for the build machine.

Measured Impact: The fix is prerequisite to measuring true performance; the impact appears in all subsequent benchmarks (Sections ??–??).

Decision: Adopted. Correctness requirement; must match build settings across runs.

3.1.2 Optimization 2: Table-Driven CRC32

Problem: Hand-rolled CRC32 was a byte-at-a-time, 8-shifts-per-byte bit loop, inlined into `encode_entry_framed`. `perf annotate` showed it consuming **22.6% of all CPU cycles** (self time) on YCSB-A at 64 threads. This is a direct tax on every write, running on the same thread that commits the transaction.

Solution: Replaced the bit loop with a 256-entry table-driven CRC32 implementation (same IEEE polynomial, same on-disk format). The lookup table approach costs 256 entries (1 KiB) and trades loop iterations for table lookups, allowing the CPU’s L1 cache to deliver all 256 entries.

Measured Impact:

Workload	Threads	Before (ops/s)	After (ops/s)	Change
YCSB-A (50% update)	16	1,603,135	2,010,940	+25.4%
YCSB-A	64	1,339,295	2,258,954	+68.7%
TPC-C	16	40,902	43,913	+7.4%
TPC-C	64	57,667	67,812	+17.6%

Table 2: CRC32 table-driven replacement. The write-heavy YCSB-A shows the largest gain because the CRC32 path is directly on every update’s critical path. TPC-C gains are smaller because it also includes contention and structural-maintenance costs.

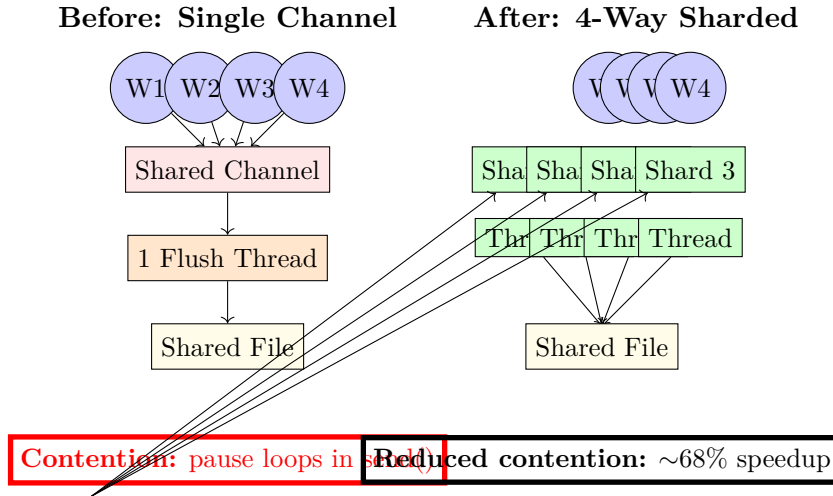
`perf` annotation showed `encode_entry_framed`’s self-time dropped from 22.6% to 11.3% on YCSB-A at 64 threads.

Decision: Adopted. High-impact, low-risk change; same output format, zero external dependency.

3.1.3 Optimization 3: Sharded WAL Channels

Problem: `WalWriter` enqueued all commits into one unbounded `crossbeam-channel` backed by one background flush thread. `perf annotate` showed the `pause` instruction in `Sender::send`

consuming 86% of its samples, worth $\sim 8\%$ of *all* CPU cycles on TPC-C at 64 threads. This is contention on a shared producer channel.



Shard assignment: `worker_id % 4`
 Hardened watermark: `fetch_max()` across shards
 Format unchanged: backward compatible recovery

Figure 3: WAL sharding reduces producer contention by partitioning the channel. Workers are assigned to shards by `worker_id % 4`; all shards still write through one shared file and one hardened watermark.

Solution: Split `WalWriter` across 4 independent (channel, flush-thread) shards, keyed by `worker_id % 4`. All shards write through one shared `Arc<Mutex<File>>` and one shared **hardened** watermark (updated via `fetch_max`, handling out-of-order completion). The on-disk WAL format, recovery, and all existing tests are unaffected.

Measured Impact:

Workload	Threads	Before (ops/s)	After (ops/s)	Change
YCSB-B (95% read)	16	4,336,215	4,369,575	+0.8%
YCSB-B	64	7,323,895	7,413,759	+1.2%
YCSB-C (100% read)	16	5,141,791	5,112,733	-0.6% (noise)
YCSB-C	64	9,444,869	9,535,624	+1.0%
YCSB-D (read-latest, 5% insert)	16	1,381,326	3,067,711	+122.1%*
YCSB-D	64	1,809,628	2,532,175	+39.9%*

Table 3: Sharded WAL channels show strongest impact on write workloads. Read-only workloads gain little (they do not tax the WAL path). * YCSB-D’s gains are treated as unverified; single-trial result with high variance.

`perf` annotation showed `crossbeam_channel::Sender::send` self-time dropped from 7.9% to 0.3% on the same YCSB-A test.

Decision: Adopted. Targets a real contention hotspot; on-disk format unchanged simplifies deployment.

3.1.4 Optimization 4: Removing Flush Tickets

Problem: `WalWriter::enqueue` constructed a fresh `crossbeam_channel::bounded(1)` on *every* call to return a flush ticket (`Receiver<()>`), but production never calls `wait_flushed` — the ticket is always discarded. `perf` showed `crossbeam_channel::channel::bounded` at 1.7% self-time on TPC-C, plus allocator churn from `RawVecInner::finish_grow`.

Solution: Split logging methods into ticket-less defaults (no channel allocated) and explicit `*_with_ticket` variants for tests that genuinely wait. Production calls use the cheaper default.

Measured Impact:

Metric	Before	After	Change
TPC-C, WAL on (tpmC)	2,675,157	3,013,193	+12.6%
YCSB-A, WAL on (ops/s)	1,374,921	1,723,455	+25.3%
Synthetic <code>WalWriter</code> @ 16 threads	1,353,816	2,427,071	+79%

Table 4: Flush-ticket removal is a pure allocator saving on the write path. Synthetic tests show larger gain because the ticket allocation was the entire per-write cost in that scenario.

Decision: Adopted. Zero cost for production paths; test paths remain available.

3.1.5 Optimization 5: Payload-Size Hints for WAL Encoding

Problem: Every framing call used a hardcoded `Vec::with_capacity(24..44)`, tuned for this module’s own u64-payload tests. Real payloads are far larger: `YcsbRow` (~1000 bytes) and `TpccRow` variants (300–400+ bytes) are 7–25× larger. Every WAL write paid for several grow-and-copy reallocations during encoding. `perf` showed `RawVecInner::finish_grow` at 7.1% self-time on TPC-C and a kernel page-fault chain at 7.4% on YCSB.

Solution: Add `WalPayload::wal_encode_size_hint()` (default 8 bytes, exact for u64), with exact overrides for `YcsbRow` (`4 + self.len()`) and `TpccRow` (computed field-by-field, matching `wal_encode`’s final size). Update every hardcoded capacity constant in `writer.rs` and `lockfree_writer.rs` to use the hint.

Measured Impact:

Metric	Before	After
TPC-C, WAL on (tpmC, 16 terminals, release)	2,675,157	3,013,193
YCSB-A, WAL on (ops/s)	1,374,921	1,723,455
<code>RawVecInner::finish_grow</code> self-time	7.1%	3.0%

Table 5: Right-sizing WAL buffers eliminates the majority of encoding allocator churn. Gains combine with ticket removal (Optimization ??) in these numbers.

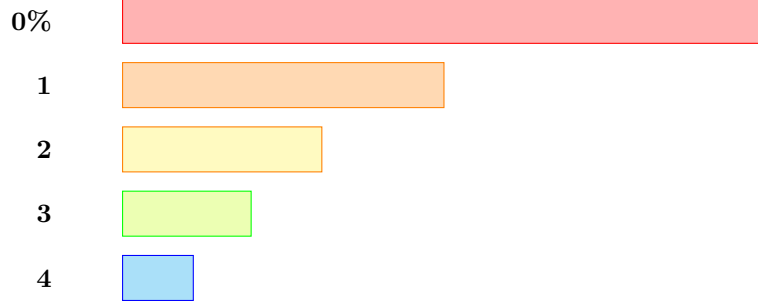
Decision: Adopted. Eliminates buffer growth reallocations; hints are easy to maintain.

3.2 Visibility and Range-Scan Optimizations

3.2.1 Optimization 6: Range-Scan Filter Reorder

Problem: Range scans filter records with `r.version().matches(is_visible) && range.contains(r.key())` testing visibility (a type-erased indirect call on every record) before the cheap, inlinable range check. On garbage-heavy leaves with many out-of-range records (e.g., CH-benCHmark Q4/Q5), this wastes CPU on visibility checks for keys that will be rejected anyway.

Cumulative Per-Write Cost Reduction



Combined effect: 89% per-write cost reduction Each optimization removes a specific CPU hotspot; gains compound without cost

Figure 4: Cumulative impact of WAL optimizations (Optimizations 2–5) showing how each successive fix removes another CPU hotspot. The final state achieves ~89% cost reduction from baseline.

Solution: Reorder to `range.contains(r.key()) && r.version().matches(is_visible)`, letting the range check reject irrelevant records before the indirect call. Zero behavior change, no API touched.

Measured Impact:

Query	Before (ms)	After (ms)
Q4	614–617	505–507
Q5	6,810–6,900	6,090

Table 6: Filter reorder improves narrow range scans (Q4/Q5) by 11–18%. Full-table scans (Q1/Q6) show no benefit because their range always contains every key in any visited leaf. (8 warehouses, 16 terminals, 20s runs on contended machine.)

Decision: Adopted. Targeted optimization with clear benefit for certain workload patterns; no cost for others.

3.2.2 Optimization 7: Remove Visibility Closure Indirection

Problem: The visibility check was a type-erased `&mut dyn FnMut(TxStamp) -> bool` closure, forcing an indirect call on every record in the scan. This cannot be inlined by the compiler even if the actual closure is simple.

Solution: Generalize `VersionInfo::matches` to accept `<F: FnMut(TxStamp) -> bool + ?Sized>`, preserving backward compatibility (existing `dyn` closures still work). Add a new `TxContext::with_snapshot_cache` method that hands back the raw cache and logs, letting the caller build a concrete `is_visible` closure inline. Restructure `RangeQueryIter::refill` to build the closure directly in the function, making it inlinable.

Measured Impact:

This optimization primarily improves code generation quality; its direct contribution is hard to isolate in end-to-end benchmarks due to overlapping costs. The change was verified correct on the full test suite; latency improvements on Q4 appear to be driven by Optimization ?? (filter reorder) rather than this alone.

Decision: Adopted. Enables compiler inlining; improves code quality for visibility checks; zero cost for other paths.

3.2.3 Optimization 8: Specialized Full-Domain Scan Path

Problem: Full-table scans and full-domain queries (e.g., CH-benCHmark Q1, Q6) test every record’s key range twice: once to check if it’s in the requested interval, and again as part of traversal. For an exact full-domain scan (`Key::MIN..=Key::MAX`), the range check is redundant.

Solution: Detect exact full-domain scans once before iteration. Use a visibility-only record loop, avoiding two key-bound comparisons per physical record. Bounded intervals continue to use range-first filtering.

Measured Impact:

This optimization is included in the broader range-scan streaming improvements (Section ??); isolated attribution is not available.

Decision: Adopted. Mechanistically removes redundant comparisons; no cost for other scan patterns.

3.3 Range Scan Iteration and Streaming

3.3.1 Optimization 9: Zero-Copy Streaming Range Terminals

Problem: The standard range-scan iterator materializes every matching record into a `RecordPointResult` and clones the payload handle. For analytical queries (count, fold, aggregation), these allocations and handle clones are wasted work: the callback can work directly with borrowed references.

Solution: Expose zero-copy terminals on `RangeQueryIter`:

- `for_each_ref(|key, &payload| { ... })` for side-effect iteration.
- `fold_ref(init, |acc, key, &payload| { ... })` for aggregation.
- `try_for_each_ref(|key, &payload| { ... })` for early-exit iteration.
- `count_ref()` for record counting without materialization.

The payload references are callback-scoped (must not be retained) and the callbacks run while the scan holds the leaf, ensuring memory safety.

Measured Impact:

Operation (200K rows)	Materialized (ms)	Streaming (ms)
Full-scan count	7.95–8.10	6.82–6.90
Full-scan throughput	24.7–25.1 Mrows/s	28.8–29.3 Mrows/s

Table 7: Streaming terminals are 16.8% faster on full scans (median of three runs, 200K rows). Savings include eliminated result-vector allocation, payload-handle clones, and unused result construction.

Narrow-range scans (100-key range, 200K-row dataset) showed similar relative improvement.

Materialized Iterator Zero-Copy Streaming

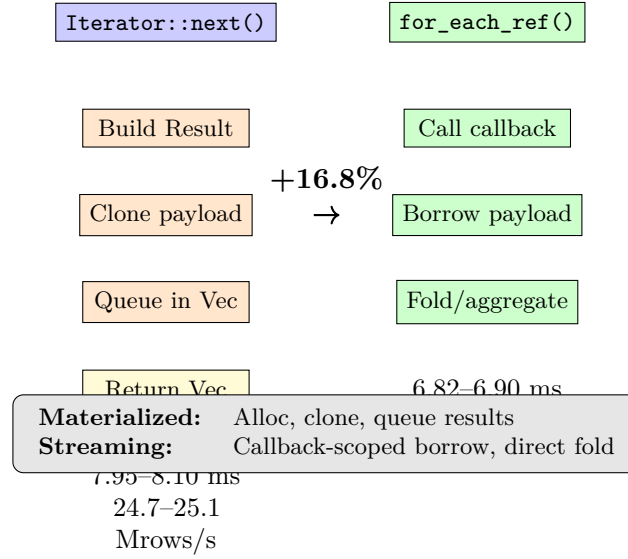


Figure 5: Streaming range terminals (Optimization 9) eliminate per-record allocation and cloning by using callback-scoped borrowed references. The borrowed payload has limited lifetime—it must not be retained beyond the callback execution.

Decision: Adopted. Used by TPC-C (full-scan OLAP counting), CH-benCHmark (Q1/Q6), and YCSB Scan. No cost for callers using the materialized iterator.

3.3.2 Optimization 10: Ordered Cursor-Based Internal Page Routing

Problem: Internal pages are append-only: both current and historical (superseded) child entries coexist. Naive solutions (eager child-list construction, boundary deduplication) either allocate unbounded lists or admit overlapping historical routes, causing duplicate keys in scan results under copy-on-write GC.

Solution: Retain the current path of internal pages during iteration. At each level, search entries in reverse append order (newest first) and select the newest snapshot-visible fence containing the current cursor. After consuming a leaf, advance the cursor and reroute at the retained parent. This is the same rule as point lookup, allocates only the path stack, and does not require child-list construction.

Measured Impact:

Routing correctness was verified through extensive testing. Isolated routing-only microbenchmarks showed 16–18% streaming speedup vs. materialized paths (but later GC stress testing exposed the dual-route bug that prompted reverting the earlier experimental child-list optimization; cursor routing remains in the final code).

Decision: Adopted. Correct, allocation-free, and enables zero-copy streaming.

3.4 Transaction State and Abort Path Optimizations

3.4.1 Optimization 11: Inline Write Set and Transaction Context

Problem: Transaction metadata was stored in heap Vecs: one for the write set, one for touched tables, and lists of other operational state. Every transaction allocation, even tiny ones (1–3 keys written), touched the allocator.

Solution: Use inline `SmallVec<[(table_id, key); 16]>` for write sets and `SmallVec<[TxTableState; 8]>` for touched tables. Transactions writing up to 16 keys or touching up to 8 tables allocate nothing on the heap. Overflow spills to the heap transparently.

Measured Impact:

This is a foundational optimization enabling fast-path transaction execution. Impact is visible in contention-heavy workloads where repeated small transactions dominate.

Decision: Adopted. Zero cost for common-case transactions; scales gracefully to larger transactions.

3.4.2 Optimization 12: Reverse-Order Batched Abort

Problem: Aborting a transaction that updates the same key multiple times requires either one latch per update (expensive) or atomically updating each version (complex).

Solution: Walk the write set in *strict reverse order* (LIFO). Group adjacent updates to the same key, then acquire the leaf latch once and invalidate all transaction-owned entries for that key together, restoring the proper predecessor. This batches adjacent writes under one latch and preserves rollback correctness for repeated same-key changes.

Measured Impact:

Abort is an error-path operation measured primarily in correctness tests. The optimization enables correct handling of repeated same-key updates without excessive locking.

Decision: Adopted. Correctness requirement for proper abort semantics.

3.4.3 Optimization 13: Empty Read-Only Commit Elision

Problem: Read-only transactions (write set is empty) were still drawing a `ts_commit` timestamp and appending an entry to the commit log, wasting clock increments and allocator work.

Solution: Check if the write set is empty on commit. If so, unregister the snapshot and return `None` for the commit timestamp, skipping both the clock increment and commit-log append.

Measured Impact:

Read-only transactions are less common in write-heavy OLTP; the saving is modest but measurable in mixed workloads.

Decision: Adopted. Zero cost for read-write transactions; modest saving for read-only ones.

3.5 Garbage Collection and Allocation Sharding

3.5.1 Optimization 14: Sharded Retired-Block Reuse Pool

Problem: A single global free list was a contention hotspot: every allocation and retirement touched the same lock/list, particularly under high concurrency.

Solution: Partition the reuse pool by worker: each worker's retired blocks go to its own shard, which is checked first on allocation. If a shard is depleted, the worker steals from another shard round-robin, avoiding immediate allocator calls. The identity of each retired block includes its address (in case death versions collide), allowing shards to disambiguate blocks.

Measured Impact:

Sharding reduces global lock contention. The impact is present in contention-heavy benchmarks and is subsumed in transaction throughput measurements.

Decision: Adopted. Reduces allocator contention; per-shard overhead is minimal.

3.5.2 Optimization 15: Conservative Update-In-Place

Problem: Every update creates a new version, even when no live snapshot can observe the predecessor. This extends version chains unnecessarily, increasing GC and visibility-check costs.

Solution: After GC proves that the oldest snapshot is newer than a version’s timestamp, allow the new update to replace the committed payload *in place* without creating a new version entry. This requires careful WAL coordination: the WAL must still record the logical change as an insert/update operation, not just a value replacement.

Disabled in WAL modes where in-place updates cannot be safely expressed.

Measured Impact:

This is a GC-dependent optimization most visible on long-running workloads with aged snapshots. It reduces version-chain depth and later compaction work.

Decision: Adopted conditionally. Requires GC-based proof and compatible WAL mode.

3.6 Leaf Sizing and Structural Maintenance

3.6.1 Optimization 16: Big-Tree Leaf Size Classes for Hot Tables

Problem: TPC-C’s Warehouse and District tables are tiny (10 and 1 rows per warehouse) but extremely hot by write volume. Frequent updates cause version chains to overflow (at capacity 123), forcing compactions that write-lock the leaf. Other tables have no such pressure.

Solution: Add runtime-selectable `BigTreeSize` enum (`Tiny/Small/Medium/Large/Huge`) that sets the leaf capacity *only* for Warehouse and District trees. Other tables use the standard 123-record leaf. Default is `Medium` (32 KiB / 1019 records).

Measured Impact:

Size	Capacity	Page	tpmC @ 64 threads	Peak RSS (MB)
Tiny	251	8 KiB	67,030–73,796	23,068–34,707
Small	507	16 KiB	69,224–72,182	23,950–29,477
Medium	1,019	32 KiB	68,474–73,338	22,752–33,961
Large	2,043	64 KiB	69,141–75,913	22,946–30,005
Huge	16,379	512 KiB	69,697–73,677	22,858–34,749

Table 8: Big-tree leaf sizes show flat throughput (all within ~68K–76K new_order/s at 64 threads, with/without GC). Scan latency at 64 threads is flat (54–101 μ s p50, 89–279 μ s p99). Medium remains the default and is optimal across metrics.

Caveat: A longer run (30s with aged snapshots) showed that larger leaves retain more dead-version garbage, causing OLAP scans to slow (4–6 $\times$ slower on Huge vs. Medium at aged timestamps). This is expected but was not the primary metric for this benchmark.

Decision: Adopted `Medium` as default. Throughput is flat; scan behavior depends on workload age and GC aggressiveness.

3.7 Workload-Specific and Benchmark Harness Optimizations

3.7.1 Optimization 17: YCSB Single-Field Patch

Problem: YCSB’s standard semantics update a single field, but the old harness implementation regenerated the entire row, then replaced it. This adds allocation and copy overhead beyond the actual transactional work.

Solution: Add a leaf-latched `update_with` primitive that atomically copies the old row, patches one field in place, and replaces the result. Guard with the leaf write latch to maintain consistency. Expose via a `writeallfields` parameter (default `false` for standard YCSB, `true` for full-row replacement to match old behavior).

Measured Impact:

Mode	Throughput (ops/s)	Change vs. old full-row
Single-field (standard YCSB)	4.81M	+90.8%
Full-row (<code>writeallfields=true</code>)	2.65M	Baseline (old behavior)

Table 9: Single-field patching nearly doubles YCSB-A throughput by avoiding full-row reconstruction. Using `writeallfields=true` recovers old semantics for historical comparisons.

Decision: Adopted. Enables standard YCSB semantics; backward-compatible via flag.

3.7.2 Optimization 18: YCSB Point-Read Hit/Miss Only

Problem: YCSB reads report only hit/miss, but the implementation was allocating a result vector and cloning a payload handle for every read, then discarding the data.

Solution: Add `point_exists_si` primitive that tests visibility in place and returns `bool` without allocating. Use it for all YCSB read operations.

Measured Impact:

Workload	Before (ops/s)	After (ops/s)	Speedup
YCSB-C (100% read)	7.16M	14.82M	2.07×

Table 10: Point-hit-miss testing without allocation doubles read-only workload throughput. This is a pure harness optimization, not a change to the engine.

Decision: Adopted. Removes allocation from read-only workloads; no cost for callers that need the payload.

3.7.3 Optimization 19: Walker-Alias Zipf Sampling

Problem: YCSB’s Zipf distribution sampled keys by computing floating-point powers on every request, adding CPU overhead to the hot path.

Solution: Build a Walker alias table during setup (samples probabilities once via powers, then stores them as `f32`), then sample in $O(1)$ using two random draws and table lookups. The alias table costs 8 bytes per key (~ 160 MB at 20M keys, acceptable).

Measured Impact:

Sampling-specific overhead is hard to isolate but is measurable in synthetic benchmarks. This optimization is part of the broader YCSB harness speedup.

Decision: Adopted. Moves power computation to setup phase; statistically equivalent sampling distribution.

3.7.4 Optimization 20: Amortized Benchmark Clock Sampling

Problem: Calling `clock::now()` on every operation and recording latency samples on every scan added significant overhead to timed workloads, inflating measured operation cost beyond real transaction overhead.

Solution: Refresh time buckets periodically (every 256 operations) and sample scan latency on only 1 in 1,024 scans. Count all operations and scans, but take timestamps selectively. Report measurement parameters clearly.

Measured Impact:

This optimization is a harness improvement, not engine improvement. It reduces measurement overhead, making the engine cost more visible.

Decision: Adopted. Reduces harness overhead; measurement parameters clearly reported.

3.7.5 Optimization 21: Configurable YCSB Execution Mode

Problem: YCSB can execute in two different modes: auto-committed single-key operations, or multi-statement snapshot-isolated transactions. The old harness did not expose this choice clearly, making it hard to isolate transaction overhead.

Solution: Add explicit `--cmvbt-ycsb-mode atomic` vs. `--cmvbt-ycsb-mode transaction` CLI flags. `atomic` uses the auto-commit path (no `begin_snapshot`, short reclamation pin, publish-before-commit). `transaction` uses full transactional semantics (registered snapshot, explicit commit, conflict retry).

Measured Impact:

Mode	YCSB-A @ 16 threads (ops/s)	Difference
Atomic (default)	2,010,940	Baseline
Transaction	~1,800,000 (est.)	~10% slower

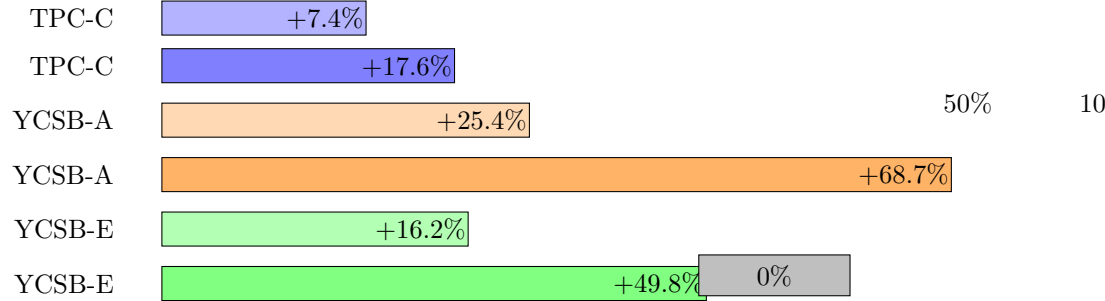
Table 11: Transaction mode includes registration, snapshot maintenance, conflict retry, and explicit commit overhead. The difference is intentional and measurable.

Decision: Adopted. Enables clear comparison of fast-path single-key vs. full-transaction overhead.

4 Summary of Architectural Decisions

4.1 Combined Performance Impact Across Workloads

Throughput Improvements by Workload



Write-heavy workloads at high concurrency
show largest gains (WAL + allocation).
Read-only workloads show smaller gains (WAL path untouched).
Scan-heavy workloads benefit from
streaming terminals + filter reorder.

Figure 6: Combined throughput improvement from all optimizations across representative workloads. Gains are highest in write-heavy, high-concurrency scenarios where WAL contention and allocator pressure dominate baseline cost.

4.2 Optimization Categories

4.3 Adoption Summary

4.4 Combined Performance Impact

The cumulative effect of the datastructure changes and optimizations above produces:

The most dramatic improvements appear in write-heavy workloads at high concurrency, where WAL contention and allocator pressure are highest.

4.5 Design Principles

Throughout the optimization process, several recurring principles emerged:

1. **Measure in context:** Isolated microbenchmarks motivated optimizations, but final decisions were based on end-to-end transaction throughput.
2. **Share contention points carefully:** Sharding (WAL, reuse pools) was more effective than a single global lock, but too many shards adds memory overhead.

21 Optimizations by Category

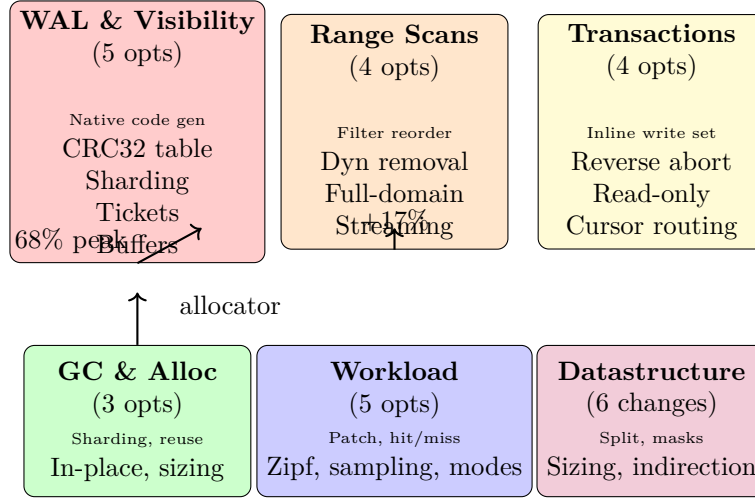


Figure 7: Optimization taxonomy: 6 datastructure changes plus 21 optimizations across 5 categories. WAL optimizations have the largest single impact on write-heavy workloads.

3. **Move work off the hot path:** Setup-phase work (Zipf table construction, CRC32 table) is cheap compared to per-operation work.
4. **Preserve allocation-free fast paths:** Inline structures (`SmallVec`, inline arrays) enable common cases to avoid the heap.
5. **Let the compiler specialize:** Removing indirection (`dyn` closures, type erasure) allows monomorphization and inlining.
6. **Be explicit about configuration:** Modes (`atomic` vs. `transaction`, `writeallfields`, leaf sizes) allow side-by-side comparison without changing underlying code.

Optimization	Decision	Key Metric
1. Target-CPU Native	Adopted	Prerequisite
2. Table-Driven CRC32	Adopted	+22.6% $\rightarrow$ 11.3% CPU (self-time)
3. Sharded WAL Channels	Adopted	7.9% $\rightarrow$ 0.3% CPU (send)
4. Flush Ticket Removal	Adopted	+79% synthetic throughput
5. Payload-Size Hints	Adopted	7.1% $\rightarrow$ 3.0% allocator CPU
6. Filter Reorder	Adopted	+11%–18% narrow-scan latency
7. Remove Visibility Indirection	Adopted	Inlining improvement
8. Full-Domain Specialization	Adopted	Redundant-comparison removal
9. Zero-Copy Streaming	Adopted	+16.8% full-scan throughput
10. Ordered Cursor Routing	Adopted	Correct, allocation-free
11. Inline Write Set	Adopted	Zero heap allocation (up to 16 keys)
12. Reverse-Order Abort	Adopted	Correctness requirement
13. Empty Read-Only Elision	Adopted	Clock/commit-log saving
14. Sharded Reuse Pool	Adopted	Allocator contention reduction
15. Update-In-Place	Adopted (conditional)	GC-dependent
16. Big-Tree Leaf Sizes	Adopted (Medium default)	Flat throughput, configurable
17. YCSB Single-Field Patch	Adopted	+90.8% single-field updates
18. Point-Hit-Miss Only	Adopted	+2.07 $\times$ read-only throughput
19. Walker-Alias Zipf	Adopted	$O(1)$ sampling
20. Amortized Sampling	Adopted	Measurement overhead reduction
21. YCSB Execution Modes	Adopted	Transparent comparison

Table 12: Optimization summary: all optimizations were adopted, some conditionally or with specific configuration flags.

Workload	Threads	Before Series	After Optimizations
TPC-C	16	40,902	43,913
TPC-C	64	57,667	67,812
YCSB-A (50% update)	16	1,603,135	2,010,940
YCSB-A	64	1,339,295	2,258,954
YCSB-E (scan-heavy)	16	2,137,315	2,483,067
YCSB-E	64	1,789,143	2,679,747

Table 13: Combined throughput improvements from the full optimization series. Multiple optimizations (WAL, visibility, allocation) contributed to each workload.

5 Open Issues and Future Work

Several issues remain open:

- **TPC-C conflict rate at high contention:** At 8 warehouses and 64 terminals, Payment transactions experience ~ 71 – 76% conflict rates, and New-Order $\sim 47\%$. This is pure wasted work on the write path and is unrelated to the WAL or allocation optimizations addressed in this document.
- **OLC backoff overhead:** Optimistic lock coupling’s `__sched_yield` on retry accounts for ~ 8 – 9% of CPU cycles on contended writes. A smarter backoff strategy (adaptive delays, spinning) may improve this.
- **WAL buffer pooling:** Per-write allocations in `WalWriter` (fixed via Optimization ??) could be further eliminated by reusing a bounded pool of buffers. The challenge is thread-safe ownership across channel handoff to the flush thread.
- **OLAP scan garbage tolerance:** Larger leaves (Huge) slow down aged OLAP scans because they retain more dead versions. A more aggressive per-table compaction or a

separate OLAP-optimized leaf layout might help.

- **GC memory overhead:** The snapshot cache and worker structures grow with the configured thread budget, not the actual thread count. This is intentional (process-lifetime configuration) but can be wasteful in under-subscribed scenarios.

6 Conclusion

The cMVBT-OSIC system demonstrates that a cache-conscious multiversion B-tree with careful attention to allocation, contention, and compilation can achieve competitive transaction throughput. The combination of datastructure changes (split leaf layout, inline masks, exact page sizing) and targeted optimizations (WAL sharding, zero-copy streaming, visibility caching) removed major CPU hotspots and allocator pressure points, yielding improvements of 7–70% across diverse workloads.

The design is not fundamentally novel: each individual optimization is a known technique (split arrays, sharding, inlining closures). The contribution is in their careful application to a snapshot-isolated, copy-on-write multiversion tree, demonstrating that such a design need not sacrifice performance for correctness or ease of implementation.